Name : Abbas Fazal.

Reg Id : BF25NWELE0713.

Assignment : 2nd.

Submitted To : Sir Rizwan.

Subject : Computer Programming.

Semester : 2nd.

Section : Power.

1.

Write a program to:

* Take 10 integer inputs in an array.

* Find and display:

Maximum value

Minimum value

Average.

```
#include <iostream>
using namespace std;

int main() {
    int arr[10];

    cout << "Enter 10 integers:\n";
    for(int i = 0; i < 10; i++) {
        cin >> arr[i];
    }

    int max = arr[0], min = arr[0];
    int sum = 0;

    for(int i = 0; i < 10; i++) {
        if(arr[i] > max) max = arr[i];
        if(arr[i] < min) min = arr[i];
        sum += arr[i];
```

```
    }

    double avg = sum / 10.0;

    cout << "Maximum = " << max << endl;
    cout << "Minimum = " << min << endl;
    cout << "Average = " << avg << endl;

    return 0;
}
```

2.

Write a program that:

* Uses a function to reverse a 1D array.

* Display the array before and after reversal

```
#include <iostream>
using namespace std;

void reverseArray(int arr[], int size) {
    for(int i = 0; i < size/2; i++) {
        int temp = arr[i];
        arr[i] = arr[size - 1 - i];
        arr[size - 1 - i] = temp;
    }
}
```

```
void display(int arr[], int size) {
    for(int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int arr[10];

    cout << "Enter 10 numbers:\n";
    for(int i = 0; i < 10; i++) {
        cin >> arr[i];
    }

    cout << "Before Reverse: ";
    display(arr, 10);

    reverseArray(arr, 10);

    cout << "After Reverse: ";
    display(arr, 10);

    return 0;
}
```

3.

Write a program to:

* Input two 3 × 3matrices.

* Perform matrix addition.

* Display the result matrix

```
#include <iostream>
using namespace std;

int main() {
    int A[3][3], B[3][3], C[3][3];

    cout << "Enter Matrix A:\n";
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            cin >> A[i][j];
        }
    }

    cout << "Enter Matrix B:\n";
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            cin >> B[i][j];
        }
    }
```

```
    // Addition
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    cout << "Result Matrix:\n";
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
            cout << C[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

4.

Write a program that:

* Stores names of 5 students in a string array.

* Sorts them in alphabetical order.

* Displays the sorted list.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
   string names[5];

   cout << "Enter 5 student names:\n";
   for(int i = 0; i < 5; i++) {
      cin >> names[i];
   }

   // Sorting (Bubble Sort)
   for(int i = 0; i < 5 - 1; i++) {
      for(int j = 0; j < 5 - i - 1; j++) {
         if(names[j] > names[j+1]) {
            string temp = names[j];
            names[j] = names[j+1];
            names[j+1] = temp;
         }
      }
```

```
    }

    cout << "Sorted Names:\n";
    for(int i = 0; i < 5; i++) {
        cout << names[i] << endl;
    }

    return 0;
}
```

5.

Power System Load Analysis (1D Array + Function)

In a power system, hourly load demand (in MW) is recorded.

Write a program to:

* Store load data for 24 hours in an array.

* Create functions to calculate:

Total load

Peak load

```
#include <iostream>
using namespace std;

double totalLoad(double arr[], int size) {
    double sum = 0;
    for(int i = 0; i < size; i++) {
        sum += arr[i];
```

```
    }
    return sum;
}

double peakLoad(double arr[], int size) {
    double max = arr[0];
    for(int i = 1; i < size; i++) {
        if(arr[i] > max) max = arr[i];
    }
    return max;
}

int main() {
    double load[24];

    cout << "Enter load for 24 hours:\n";
    for(int i = 0; i < 24; i++) {
        cin >> load[i];
    }

    cout << "Total Load = " << totalLoad(load, 24) << endl;
    cout << "Peak Load = " << peakLoad(load, 24) << endl;

    return 0;
}
```

6.

Design a program to:

Store student names (string array) and their marks (1D array).

Use functions to:

Find topper

Display pass/fail status

Search a student by name

```
#include <iostream>
#include <string>
using namespace std;

int findTopper(int marks[], int size) {
    int index = 0;
    for(int i = 1; i < size; i++) {
        if(marks[i] > marks[index]) {
            index = i;
        }
    }
    return index;
}

void passFail(int marks[], int size) {
    for(int i = 0; i < size; i++) {
        if(marks[i] >= 50)
            cout << "Pass\n";
        else
```

```
        cout << "Fail\n";
    }
}

void searchStudent(string names[], int marks[], int size, string key) {
    for(int i = 0; i < size; i++) {
        if(names[i] == key) {
            cout << "Found! Marks = " << marks[i];
            return;
        }
    }
    cout << "Student not found";
}

int main() {
    string names[5];
    int marks[5];

    cout << "Enter names and marks:\n";
    for(int i = 0; i < 5; i++) {
        cin >> names[i] >> marks[i];
    }

    int topper = findTopper(marks, 5);
    cout << "Topper: " << names[topper] << endl;

    cout << "\nPass/Fail Status:\n";
    passFail(marks, 5);
```

```
    string search;
    cout << "\nEnter name to search: ";
    cin >> search;

    searchStudent(names, marks, 5, search);

    return 0;
}
```

7.

Renewable Energy Monitoring System (1D Array + Functions)

A solar power plant records the energy generated (in kWh) every hour for one day.

Write a C++ program to:

* Store energy generation data for 24 hours in a 1D array.

Create functions to:

* Calculate total energy generated in a day

* Find the hour with maximum generation

* Calculate average energy generation

* Display all results clearly.

```
#include <iostream>
using namespace std;

// Function to calculate total energy
double totalEnergy(double arr[], int size) {
```

```
    double sum = 0;
    for(int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return sum;
}

// Function to find hour with maximum generation
int maxHour(double arr[], int size) {
    int index = 0;
    for(int i = 1; i < size; i++) {
        if(arr[i] > arr[index]) {
            index = i;
        }
    }
    return index;
}

// Function to calculate average
double averageEnergy(double arr[], int size) {
    return totalEnergy(arr, size) / size;
}

int main() {
    double energy[24];

    cout << "Enter energy generated for 24 hours:\n";
    for(int i = 0; i < 24; i++) {
        cin >> energy[i];
```

```
    }

    double total = totalEnergy(energy, 24);
    int max_index = maxHour(energy, 24);
    double avg = averageEnergy(energy, 24);

    cout << "\nTotal Energy = " << total << " kWh";
    cout << "\nMaximum Generation at Hour = " << max_index;
    cout << "\nAverage Energy = " << avg << " kWh\n";

    return 0;
}
```

8.

Battery Health Monitoring (1D Array + Functions)

In battery management systems, voltage readings are monitored to ensure safe operation.

Write a program to:

* Store 20 voltage readings of a battery in an array.

* Define safe voltage limits (e.g., 3.0V to 4.2V).

* Create functions to:

* Count how many readings are below or above safe limits

* Find the maximum and minimum voltage

* Display a warning message if unsafe readings exist

```
#include <iostream>
using namespace std;

int countUnsafe(float arr[], int size, float low, float high) {
```

```
    int count = 0;
    for(int i = 0; i < size; i++) {
        if(arr[i] < low || arr[i] > high) {
            count++;
        }
    }
    return count;
}

void findMinMax(float arr[], int size, float &minV, float &maxV) {
    minV = maxV = arr[0];

    for(int i = 1; i < size; i++) {
        if(arr[i] < minV) minV = arr[i];
        if(arr[i] > maxV) maxV = arr[i];
    }
}

int main() {
    float voltage[20];
    float low = 3.0, high = 4.2;

    cout << "Enter 20 voltage readings:\n";
    for(int i = 0; i < 20; i++) {
        cin >> voltage[i];
    }

    float minV, maxV;
    findMinMax(voltage, 20, minV, maxV);
```

```
    int unsafe = countUnsafe(voltage, 20, low, high);

    cout << "\nMinimum Voltage: " << minV;
    cout << "\nMaximum Voltage: " << maxV;
    cout << "\nUnsafe Readings: " << unsafe;

    if(unsafe > 0) {
        cout << "\nWarning: Unsafe voltage detected!";
    }

    return 0;
}
```

9.

Smart Grid Load Distribution (2D Array + Functions)

In a smart grid, electricity consumption is recorded across multiple regions.

Write a program to:

* Store power consumption (in MW) for 4 regions over 7 days using a 2D array.
* Create functions to:
* Calculate total consumption for each region
* Identify the day with highest total demand
* Compute overall average consumption.

```
#include <iostream>
using namespace std;

void totalPerRegion(float arr[4][7]) {
```

```
    for(int i = 0; i < 4; i++) {
        float sum = 0;
        for(int j = 0; j < 7; j++) {
            sum += arr[i][j];
        }
        cout << "Region " << i+1 << " Total = " << sum << " MW\n";
    }
}

int highestDay(float arr[4][7]) {
    float dayTotal[7] = {0};

    for(int j = 0; j < 7; j++) {
        for(int i = 0; i < 4; i++) {
            dayTotal[j] += arr[i][j];
        }
    }

    int index = 0;
    for(int i = 1; i < 7; i++) {
        if(dayTotal[i] > dayTotal[index]) {
            index = i;
        }
    }
    return index;
}

float overallAverage(float arr[4][7]) {
    float sum = 0;
```

```
    for(int i = 0; i < 4; i++) {
        for(int j = 0; j < 7; j++) {
            sum += arr[i][j];
        }
    }
    return sum / (4 * 7);
}

int main() {
    float grid[4][7];

    cout << "Enter power consumption for 4 regions over 7 days:\n";
    for(int i = 0; i < 4; i++) {
        for(int j = 0; j < 7; j++) {
            cin >> grid[i][j];
        }
    }

    cout << "\nTotal Consumption per Region:\n";
    totalPerRegion(grid);

    int day = highestDay(grid);
    cout << "\nDay with Highest Demand = Day " << day + 1;

    cout << "\nOverall Average Consumption = " << overallAverage(grid) << " MW\n";

    return 0;
}
```

10.

Classroom Attendance System (2D Array + Functions)

Teachers often track student attendance daily.

Write a program to:

* Store attendance for 5 students over 7 days using a 2D array

(Use 1 for present, 0 for absent).

* Create functions to:

* Calculate total attendance of each student

* Find the student with highest attendance

* Calculate overall class attendance percentage

```
#include <iostream>
using namespace std;

void totalAttendance(int arr[5][7]) {
    for(int i = 0; i < 5; i++) {
        int sum = 0;
        for(int j = 0; j < 7; j++) {
            sum += arr[i][j];
        }
        cout << "Student " << i+1 << " Attendance = " << sum << endl;
    }
}

int bestStudent(int arr[5][7]) {
    int maxSum = 0, index = 0;
```

```
    for(int i = 0; i < 5; i++) {
        int sum = 0;
        for(int j = 0; j < 7; j++) {
            sum += arr[i][j];
        }

        if(sum > maxSum) {
            maxSum = sum;
            index = i;
        }
    }
    return index;
}

float classPercentage(int arr[5][7]) {
    int totalPresent = 0;

    for(int i = 0; i < 5; i++) {
        for(int j = 0; j < 7; j++) {
            totalPresent += arr[i][j];
        }
    }

    return (totalPresent / 35.0) * 100;
}

int main() {
    int attendance[5][7];
```

```
    cout << "Enter attendance (1=Present, 0=Absent):\n";
    for(int i = 0; i < 5; i++) {
        for(int j = 0; j < 7; j++) {
            cin >> attendance[i][j];
        }
    }

    cout << "\nTotal Attendance:\n";
    totalAttendance(attendance);

    int best = bestStudent(attendance);
    cout << "\nBest Student: Student " << best + 1;

    cout << "\nClass Attendance Percentage = "
         << classPercentage(attendance) << "%\n";

    return 0;
}
```